Time-Series Forecasting System

End-to-End Production ML System • US State Beverage Sales • 2024

43 / 43	4	36 / 43	24 / 24
STATES FORECASTED	MODELS COMPARED	XGBOOST WINS	TESTS PASSING

System Pipeline

01 Ingestion	02 Features	03 Models	04 Evaluation	05 API
CSV → Clean Date parser Monthly panel Train/val split	Lag: t-1…t-12 Rolling mean/std Calendar, Holiday YoY growth	SARIMA · Prophet XGBoost · LSTM 4 models/state 172 total fits	MAE · RMSE MAPE · sMAPE Auto-select best Retrain on full	FastAPI 7 endpoints Swagger UI Pydantic v2

Author: Biplaba Kr Samal	**Repository:** github.com/BiplabaKrSamal/Time-Series-Forecasting-System-with-API	**Dataset:** US Beverages Sales 2019–2023

Dataset & Data Pipeline

SECTION 01

Dataset Summary

Attribute	Value
Source	US Beverages Category Sales by State
Time Range	January 2019 – December 2023 (5 years)
Granularity	Monthly per US state
States	43 states
Raw Rows	8,084
Monthly Panel	2,580 rows (43 × 60 months)
Target Variable	total — monthly sales in USD
Train Split	2,322 rows (54 months per state)
Validation Split	258 rows (6 months per state, no leakage)

Data Cleaning Steps

Step	Challenge	Solution
Date Parsing	Two mixed formats: M/D/YYYY and DD-MM-YYYY	Regex-based _parse_date_flexible() handles both
Sales Cleaning	Comma-formatted numbers: "109,574,036"	Strip commas + whitespace → cast to float64
Deduplication	Multiple rows with same (state, date)	Drop exact duplicate (state, date) pairs
Monthly Panel	Irregular observation dates, gaps	Snap to month-end → reindex → linear interpolate
Edge NaNs	NaN at start/end after interpolation	ffill() then bfill() per state group
Train/Val Split	Risk of data leakage	Last 6 months per state → val; remainder → train

Feature Engineering

SECTION 02

All features are computed on the training series only. Rolling statistics use **.shift(1)** before **.rolling()** — this ensures the rolling window never sees the current target value, eliminating data leakage entirely.

Category	Feature Names	Purpose
Lag Features	`lag_1, lag_2, lag_3, lag_6, lag_12`	Autoregressive memory at multiple horizons
Rolling Mean	`rolling_mean_3, rolling_mean_6, rolling_mean_12`	Local trend smoothing over 3, 6, 12 months
Rolling Std	`rolling_std_3, rolling_std_6, rolling_std_12`	Volatility encoding; models uncertainty
Calendar	`month, quarter, year, trend`	Seasonal position + linear trend proxy
Cyclical Encoding	`sin_month, cos_month`	Circular month (Jan connects to Dec smoothly)
Holiday Flag	`holiday_month`	1 if any US federal holiday in that month
YoY Growth	`yoy_growth`	pct_change(12) — captures COVID/recovery shape

Critical: No-Leakage Implementation

The canonical mistake in time-series feature engineering is computing rolling statistics that include the current timestep. The correct pattern:

```
df["rolling_mean_3"] = df["sales"].shift(1).rolling(3).mean()
```

The shift(1) moves the series one step forward before rolling, so the window at time t uses only [t-3, t-2, t-1] — never t itself.

Top XGBoost Feature Importances (aggregate across all states)

Rank	Feature	Avg Importance	Interpretation
#1	`rolling_mean_3`	24.2%	Recent 3-month average dominates predictions
#2	`trend`	13.8%	Linear trend index captures long-run growth
#3	`lag_3`	11.8%	Sales 3 months ago — strong quarterly autocorrelation
#4	`yoy_growth`	9.7%	Year-over-year % — captures COVID recovery trajectory
#5	`rolling_mean_12`	7.3%	Annual rolling average smooths seasonal noise
#6	`cos_month`	6.1%	Cyclical month encoding for smooth seasonality
#7	`lag_12`	5.8%	Same month last year — strong seasonal baseline
#8	`rolling_std_3`	4.9%	Short-term volatility signal

Models

SECTION 03

SARIMA

statsmodels 0.14

- Seasonal ARIMA with automatic AIC-based order selection
- ADF test auto-determines differencing order d
- Grid search: $p,q \in \{0,1,2\} \cdot P,D,Q \in \{0,1\} \cdot m=12$
- Captures linear autoregression + annual seasonality
- Wins on states with stable, sinusoidal seasonal patterns

States won: 7 / 43 (16%) **Best at:** CA, NY, PA, VA, WA, SC, WI **Avg val MAPE:** 74.4%

Prophet

facebook/prophet 1.1.5

- Additive/multiplicative decomposition model
- Automatic changepoint detection in trend
- US federal holidays added as explicit regressors
- Changepoint prior = 0.05 (conservative overfitting guard)
- Multiplicative seasonality (sales scale with trend level)

Seasonality: Multiplicative **Changepoints:** Auto-detected **Holidays:** US federal (add_country_holidays) **Avg val MAPE:** 273.4%

XGBoost

xgboost 2.0

- Gradient-boosted trees on 16-feature engineered matrix
- Recursive walk-forward multi-step prediction strategy
- Each prediction fed back as input for next step
- L1=0.1 + L2=1.0 regularisation prevents overfitting
- Captures non-linear interactions between lag features

States won: 36 / 43 (84%) ★ **Trees / Depth:** 300 / 4 **Learning rate:** 0.05 **Avg val MAPE:** 52.5%

LSTM

TensorFlow 2.13 / Keras

- Recurrent deep learning with 12-month lookback window
- Architecture: LSTM(64) → Dropout(0.2) → Dense(32) → Dense(1)
- Huber loss for robustness against COVID-era outliers
- Early stopping (patience=15) on internal 20% val split
- Recursive decoding for multi-step horizon

Lookback: 12 months **Loss:** Huber **Optimizer:** Adam lr=0.001 **Early stop:** patience=15

Results & Model Leaderboard

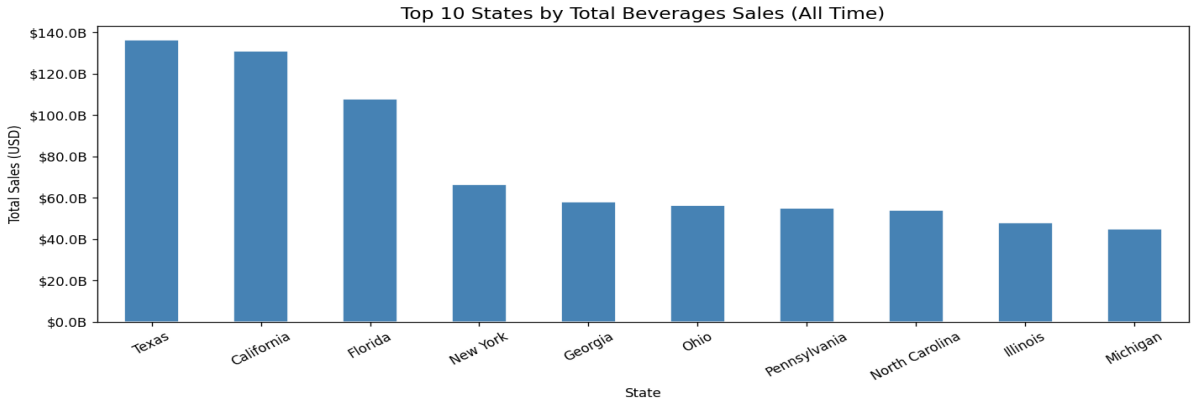
SECTION 04

Global Model Performance

XGBoost 36/43 states • avg MAPE 52.5%

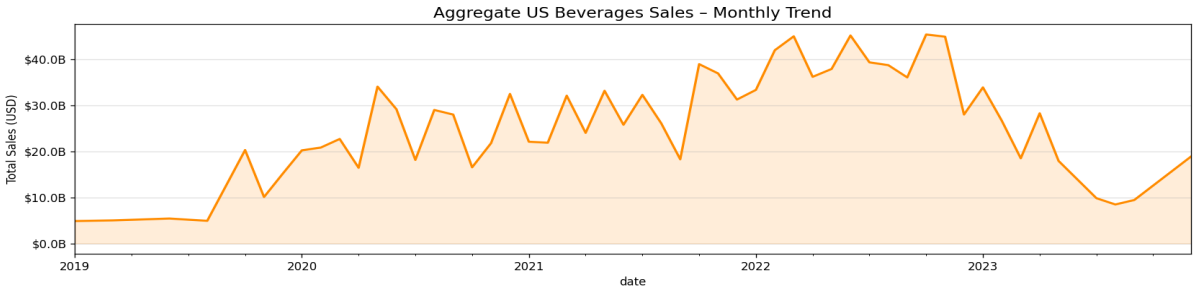
SARIMA 7/43 states • avg MAPE 74.4%

Top 10 States by Total Sales (2019–2023)



Beverage sales are highly concentrated: Texas and Florida together account for ~25% of total US sales.

National Monthly Sales Trend



Strong growth 2019–2022. Post-COVID normalisation visible in late 2023.

43-State Forecast — January & February 2024

SECTION 05

State	Best Model	Jan 2024 Forecast	Feb 2024 Forecast	Best MAPE %
Texas	XGBoost	$1.57B	$1.71B	48.4%
Florida	XGBoost	$1.46B	$1.50B	59.7%
California	SARIMA	$1.08B	$1.11B	45.9%
Georgia	XGBoost	$741.9M	$842.5M	51.6%
Ohio	XGBoost	$670.4M	$711.3M	51.5%
North Carolina	XGBoost	$648.1M	$682.9M	55.2%
New York	SARIMA	$574.7M	$516.7M	36.9%
Illinois	XGBoost	$573.3M	$604.9M	49.1%
Virginia	SARIMA	$567.9M	$699.4M	41.2%
Michigan	XGBoost	$519.6M	$579.9M	46.7%
Tennessee	XGBoost	$480.3M	$497.2M	63.2%
Pennsylvania	SARIMA	$470.6M	$446.0M	37.6%
Indiana	XGBoost	$413.3M	$455.1M	54.0%
Arizona	XGBoost	$400.6M	$440.4M	57.0%
Alabama	XGBoost	$382.0M	$434.9M	49.1%
Missouri	XGBoost	$368.1M	$393.5M	50.7%
Louisiana	XGBoost	$335.5M	$363.5M	49.1%
Massachusetts	XGBoost	$319.6M	$348.3M	45.4%
Colorado	XGBoost	$312.3M	$356.5M	56.3%
Kentucky	XGBoost	$310.0M	$323.0M	64.3%
Maryland	XGBoost	$306.4M	$317.5M	56.0%
Minnesota	XGBoost	$278.5M	$306.9M	46.4%
Oklahoma	XGBoost	$270.9M	$293.1M	61.0%
South Carolina	SARIMA	$254.9M	$272.3M	43.9%
Washington	SARIMA	$254.8M	$231.4M	41.5%
Mississippi	XGBoost	$227.4M	$251.5M	54.9%
Wisconsin	SARIMA	$210.0M	$184.4M	36.8%
Oregon	XGBoost	$208.6M	$228.3M	55.4%
Iowa	XGBoost	$204.6M	$226.8M	50.3%
Arkansas	XGBoost	$195.5M	$211.9M	55.0%
Nevada	XGBoost	$187.2M	$202.5M	63.1%
Kansas	XGBoost	$167.4M	$176.0M	49.6%
Connecticut	XGBoost	$156.7M	$174.0M	45.6%

State	Best Model	Jan 2024 Forecast	Feb 2024 Forecast	Best MAPE %
Utah	XGBoost	$153.9M	$171.3M	53.3%
West Virginia	XGBoost	$134.2M	$142.0M	57.2%
New Mexico	XGBoost	$132.9M	$139.7M	62.1%
Nebraska	XGBoost	$116.9M	$131.7M	54.4%
New Hampshire	XGBoost	$98.6M	$109.7M	40.2%
Maine	XGBoost	$83.9M	$93.7M	54.4%
South Dakota	XGBoost	$51.6M	$55.9M	45.8%
Rhode Island	XGBoost	$43.4M	$49.2M	41.1%
Wyoming	XGBoost	$39.1M	$45.2M	54.5%
Vermont	XGBoost	$35.1M	$38.3M	51.0%

Total US January 2024 Forecast: $16.01B

REST API Reference

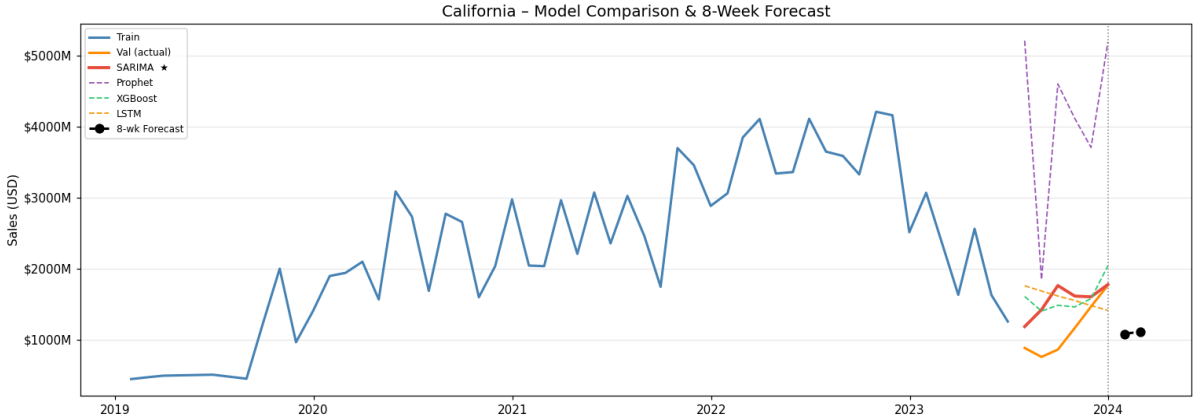
SECTION 06

Method	Endpoint	Description
GET	`/`	Health check — status, version, states loaded
GET	`/states`	List all 43 available state names
GET	`/forecast/{state}`	Best-model 8-week forecast for one state
GET	`/forecast/{state}/compare`	All-model validation metrics + forecasts
GET	`/forecast/{state}/history`	Historical sales + forecast (chart-ready)
POST	`/forecast/batch`	Batch forecasts for multiple states in one call
GET	`/models/leaderboard`	Global model win counts and average MAPE

Example Response — GET /forecast/California

```
{
  "state": "California",
  "best_model": "SARIMA",
  "forecast_steps": 2,
  "unit": "USD (monthly sales)",
  "forecast": [
    { "date": "2024-01-31", "value": 1081889233.46 },
    { "date": "2024-02-29", "value": 1113320813.50 }
  ],
  "generated_at": "2024-12-01T10:23:45.123456"
}
```

California — Model Comparison & 8-Week Forecast



SARIMA wins on California. Validation period shows all four models vs actual.

Testing & Repository Structure

SECTION 07

Test Suite — 24 / 24 Passing

Test Class	Tests	What Is Verified
TestPreprocessing	6	Date parsing (2 formats), NaN cleaning, panel construction, no-leakage split
TestFeatures	3	Feature matrix shape, zero NaN output, LSTM sequence construction
TestModelInterfaces	5	fit() + predict() contract for all 4 models; error on unfit model
TestEvaluation	3	MAPE with zero actuals, metrics dict keys, selector picks best correctly
TestAPI	7	All 7 endpoints: 200 responses, 404 on unknown state, batch, leaderboard
Total	**24**	**100% passing ✓**

Repository Structure

Path	Description
`src/preprocessing.py`	Data loading, cleaning, panel building, train/val split
`src/features.py`	All feature engineering (lag, rolling, calendar, holiday, YoY)
`src/evaluation.py`	MAE · RMSE · MAPE · sMAPE metrics + ModelSelector
`src/forecaster.py`	Full 43-state training orchestrator
`src/models/arima_model.py`	SARIMA with AIC grid-search auto-order selection
`src/models/prophet_model.py`	Facebook Prophet + US holidays
`src/models/xgboost_model.py`	XGBoost with recursive walk-forward prediction
`src/models/lstm_model.py`	LSTM + Dropout (TensorFlow/Keras), Huber loss
`api/main.py`	FastAPI — 7 REST endpoints, Swagger UI, Pydantic v2
`docs/`	ARCHITECTURE · API_REFERENCE · MODELS · RESULTS · SETUP
`tests/`	24 unit & integration tests
`notebooks/demo_walkthrough.py`	10-step annotated pipeline demo
`outputs/dashboard.html`	Self-contained interactive dashboard
`outputs/VIDEO_DEMO.html`	Full cinematic video demo (self-contained)
`outputs/forecasts/all_forecasts.json`	All 43 state forecasts + metrics
`outputs/plots/`	43 forecast charts + 5 EDA visualisations
`.github/workflows/ci.yml`	GitHub Actions CI pipeline

Quick Start Commands

```
# Install dependencies
pip install -r requirements.txt
```

```
# Run step-by-step demo
python notebooks/demo_walkthrough.py
```

```
# Train all 43 states
python run_all_states.py
```

```
# Start REST API (then open http://localhost:8000/docs)
cd api && uvicorn main:app --reload --port 8000
```

```
# Run all tests
python -m pytest tests/ -v
```